

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a65ebb15c36eb8811.jsonl`  
- SHA-256: `916a0a49c418138c5be312c7a982bf7dbc3cbf73f26e9af3ed4a7b46fb4446a4`  
- Source modified: `2026-06-10T16:25:50+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:22:08.054Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Transient keychain read failure mints a new key and silently overwrites the only DB key
SEVERITY CLAIMED: high
FILE: src/muneem/db_key.py (line 24-31)
DESCRIPTION: `secrets.get\_secret` (src/muneem/secrets.py:25-30) swallows every `KeyringError` and returns `None`, deliberately treating 'backend unavailable' the same as 'key absent'. `get\_or\_create\_key()` then interprets that `None` as 'no key has ever been minted', generates a fresh key, and `set\_secret` OVERWRITES the existing 'db-key' entry. If a read fails transiently (locked/temporarily failing Credential Manager session, keyring backend hiccup) while the subsequent write succeeds, the only copy of the real ledger key is destroyed; the next `connect\_encrypted` fails with 'wrong key' and the data is gone unless a recovery file exists. `connect\_encrypted` (src/muneem/db.py:207) calls `get\_or\_create\_key()` unconditionally ? even when an encrypted ledger file already exists on disk, which is exactly the case where minting is never the right answer.
EVIDENCE QUOTED:

```
def get_or_create_key() -> str:  
    existing = _keychain.get_secret(_DB_KEY_NAME)  
    if existing:  
        return existing  
    key = _stdlib_secrets.token_bytes(32).hex()  
    _keychain.set_secret(_DB_KEY_NAME, key)  
    return key
```


SUGGESTED FIX: Distinguish 'absent' from 'unavailable': add a get_secret variant that raises (or returns a sentinel) on KeyringError, and have get_or_create_key abort loudly instead of minting when the backend errored. Additionally, gate minting on the ledger file: if the encrypted DB file already exists but the keychain has no key, refuse with a pointer to 'muneem db recover' rather than minting a key that cannot open it.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account

number", "a client_secret value").

- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.

2. user (2026-06-10T16:22:12.999Z)

```
1      """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3      muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4      The 256-bit key is generated on first use and stored in the OS keychain via
5      :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6      never on disk in plaintext, never in the repo, never logged.
7
8      It is handed to SQLCipher in **raw-key form** (`PRAGMA key = 'x'<64 hex>'`), which
uses
9      the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10     key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11     passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12     """
13
14     from __future__ import annotations
15
16     import secrets as _stdlib_secrets
17
18     from muneem import secrets as _keychain
19
20     # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21     _DB_KEY_NAME = "db-key"
22
23
24     def get_or_create_key() -> str:
25         """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26         existing = _keychain.get_secret(_DB_KEY_NAME)
27         if existing:
28             return existing
29         key = _stdlib_secrets.token_bytes(32).hex()
30         _keychain.set_secret(_DB_KEY_NAME, key)
31         return key
32
33
34     def peek_key() -> str | None:
35         """Return the stored key, or `None` if none has been minted yet. Never creates
one."""
36         return _keychain.get_secret(_DB_KEY_NAME)
37
38
39     def new_key() -> str:
40         """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41         return _stdlib_secrets.token_bytes(32).hex()
42
43
44     def set_key(key: str) -> None:
45         """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46         _keychain.set_secret(_DB_KEY_NAME, key)
47
```

3. user (2026-06-10T16:22:14.038Z)

```
1      """Secret storage via the OS keychain.
2
```

```

3      OAuth tokens, cached statement passwords, and the DB encryption key are stored here
(Windows
4      Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5      in logs. See DESIGN.md section 5.
6
7      External-call discipline (every keyring call is wrapped with attribution):
8      - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
`None`
9      (absence is a valid state; this also keeps headless CI happy).
10     - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11     because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12     included for diagnosis; the value never is.
13     """
14
15     from __future__ import annotations
16
17     import keyring
18     from keyring.errors import KeyringError, PasswordDeleteError
19
20     from muneem.errors import KeychainError
21
22     _SERVICE = "muneem"
23
24
25     def get_secret(name: str) -> str | None:
26         """Return the stored secret, or `None` if absent or the keychain is
unavailable."""
27         try:
28             return keyring.get_password(_SERVICE, name)
29         except KeyringError:
30             return None
31
32
33     def set_secret(name: str, value: str) -> None:
34         """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35         try:
36             keyring.set_password(_SERVICE, name, value)
37         except KeyringError as exc:
38             # Name only ? never the value ? in the message (it may be a password or the DB
key).
39             raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42     def delete_secret(name: str) -> None:
43         """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44         try:
45             keyring.delete_password(_SERVICE, name)
46         except (PasswordDeleteError, KeyringError):
47             pass
48

```

4. user (2026-06-10T16:22:20.041Z)

```

1      """Local SQLite store: the `documents` and `offers` slice of the schema.
2
3      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4      (accounts, transactions, holdings, merchants, items) grows with later milestones.
5      `documents.sha256` is UNIQUE so re-ingesting the same file is a no-op.
6      `offers.source` records provenance ('text' = exact text layer, 'ocr' = approximate).

```

```

7      """
8
9      from __future__ import annotations
10
11     import os
12     import sqlite3
13     from collections.abc import Iterator
14     from contextlib import contextmanager
15     from pathlib import Path
16
17     from muneem.errors import EncryptedDatabaseError # re-exported so
db.EncryptedDatabaseError works
18
19     SCHEMA_VERSION = 4
20
21     # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
22     # file does not, so we can tell them apart without a key.
23     _SQLITE_MAGIC = b"SQLite format 3\x00"
24
25     _SCHEMA = """
26 CREATE TABLE IF NOT EXISTS documents (
27     id            INTEGER PRIMARY KEY,
28     source        TEXT NOT NULL,
29     external_id   TEXT,
30     sender        TEXT,
31     subject       TEXT,
32     received_date TEXT,
33     doc_type      TEXT,
34     sha256        TEXT NOT NULL UNIQUE,
35     path          TEXT,
36     status        TEXT NOT NULL DEFAULT 'parsed',
37     created_at    TEXT NOT NULL DEFAULT (datetime('now'))
38 );
39
40 CREATE TABLE IF NOT EXISTS offers (
41     id            INTEGER PRIMARY KEY,
42     document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
43     merchant      TEXT,
44     promo_code    TEXT,
45     description   TEXT,
46     expiry_raw    TEXT,
47     expiry_type   TEXT,
48     source        TEXT NOT NULL DEFAULT 'text',
49     redeemed      INTEGER NOT NULL DEFAULT 0,
50     source_text   TEXT
51 );
52
53 CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
54
55 -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
56 -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
57 -- Normalization into a unified view layer is planned for a later phase
58 -- once all bank schemas are stabilized.
59 CREATE TABLE IF NOT EXISTS axis_transactions (
60     id INTEGER PRIMARY KEY,
61     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
62     account_number TEXT,
63     txn_date TEXT,
64     value_date TEXT,
65     particulars TEXT,
66     debit REAL,
67     credit REAL,
68     balance REAL
69 );

```

```

70
71 CREATE TABLE IF NOT EXISTS axis_cc_transactions (
72     id INTEGER PRIMARY KEY,
73     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
74     account_number TEXT,
75     txn_date TEXT,
76     particulars TEXT,
77     merchant_category TEXT,
78     amount REAL,
79     type TEXT
80 );
81
82 CREATE TABLE IF NOT EXISTS au_transactions (
83     id INTEGER PRIMARY KEY,
84     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
85     account_number TEXT,
86     txn_date TEXT,
87     value_date TEXT,
88     particulars TEXT,
89     debit REAL,
90     credit REAL,
91     balance REAL
92 );
93
94 CREATE TABLE IF NOT EXISTS hdfc_transactions (
95     id INTEGER PRIMARY KEY,
96     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
97     account_number TEXT,
98     txn_date TEXT,
99     value_date TEXT,
100    particulars TEXT,
101    ref TEXT,
102    withdrawal REAL,
103    deposit REAL,
104    closing_balance REAL
105 );
106 """
107
108
109 def _raw_key_pragma(key: str) -> str:
110     """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
111     return f"PRAGMA key = \"x'{key}'\""
112
113
114 def is_plaintext_sqlite(db_path: Path | str) -> bool:
115     """True iff the file exists and begins with the *unencrypted* SQLite magic header.
116
117     Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
118     without needing the key.
119     """
120     try:
121         with open(db_path, "rb") as fh:
122             return fh.read(16) == _SQLITE_MAGIC
123     except OSError:
124         return False
125
126
127 def _harden_permissions(path: str) -> None:
128     """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
129     if path == ":memory:":
130         return
131     try:
132         os.chmod(path, 0o600)

```

```

133     except OSError:
134         pass
135
136
137 def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
138     """Open an encrypted connection via SQLCipher, verifying the key before returning.
139
140     A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
141     the
142     key is wrong (or the file is not an encrypted muneem ledger), this raises.
143     """
144     try:
145         import sqlcipher3
146     except ImportError as exc: # pragma: no cover - exercised only without the binding
147         raise EncryptedDatabaseError(
148             "database encryption requested but the 'sqlcipher3' package is not
149             installed"
150             ) from exc
151
152     conn = sqlcipher3.connect(path)
153     conn.execute(_raw_key_pragma(key)) # MUST be the first statement on the connection
154     try:
155         conn.execute("SELECT count(*) FROM sqlite_master") # forces key verification
156     except sqlcipher3.DatabaseError as exc:
157         conn.close()
158         raise EncryptedDatabaseError(
159             f"could not open encrypted database {Path(path).name}: wrong key, or not an
160             "
161             "encrypted muneem ledger"
162             ) from exc
163
164     conn.row_factory = sqlcipher3.Row
165     return conn
166
167
168 @contextmanager
169 def connect(db_path: Path | str, *, key: str | None = None) ->
170 Iterator[sqlite3.Connection]:
171     """Open a connection (commit on success, always close). Parent dirs created.
172
173     With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
174     SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
175     path for
176     ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
177     :func:`_open_encrypted`); the production ledger is opened via
178     :func:`connect_encrypted`.
179     """
180     path = str(db_path)
181     if path != ":memory:":
182         Path(path).parent.mkdir(parents=True, exist_ok=True)
183     if key is not None:
184         conn = _open_encrypted(path, key)
185         _harden_permissions(path)
186     else:
187         conn = sqlite3.connect(path)
188         conn.row_factory = sqlite3.Row
189         conn.execute("PRAGMA foreign_keys = ON")
190     try:
191         yield conn
192         conn.commit()
193     finally:
194         conn.close()
195
196
197 @contextmanager
198 def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:

```

```

192     """Open the real on-disk ledger, encrypted with the keychain-managed key.
193
194     The production path: resolves (or mints, on first use) the 256-bit key from the OS
195     keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196     file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197     "file is not a database" error.
198     """
199     from muneem.db_key import get_or_create_key
200
201     path = str(db_path)
202     if is_plaintext_sqlite(path):
203         raise EncryptedDatabaseError(
204             f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205             "to migrate it to an encrypted ledger"
206         )
207     with connect(path, key=get_or_create_key()) as conn:
208         yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.
213
214         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
215         securely delete (it still holds the cleartext data). Returns the number of user
tables
216         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
217         """
218         import sqlcipher3
219
220         path = Path(db_path)
221         if not is_plaintext_sqlite(path):
222             raise EncryptedDatabaseError(
223                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
224             )
225
226         enc_tmp = path.with_name(path.name + ".enc.tmp")
227         enc_tmp.unlink(missing_ok=True)
228         try:
229             src = sqlcipher3.connect(str(path)) # open the plaintext source (no key)
230             try:
231                 src_tables = src.execute(
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/") # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY '{key}'")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241         # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242         # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243         # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244         with connect(str(enc_tmp), key=key) as check:
245             integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246             ntables = check.execute(

```

```

247         "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248     ).fetchone()[0]
249     if integrity != "ok" or ntables != src_tables:
250         raise EncryptedDatabaseError(
251             f"encryption migration verification failed for {path.name} "
252             f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
253         )
254
255     backup = path.with_name(path.name + ".plaintext.bak")
256     backup.unlink(missing_ok=True)
257     path.rename(backup)
258     enc_tmp.rename(path) # consumed on success
259     _harden_permissions(str(path))
260     finally:
261         # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
262         enc_tmp.unlink(missing_ok=True)
263     return int(ntables)
264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \"{new_key}\"")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300                 source.backup(target) # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:

```



```

304         source.close()
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
310         DB).
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
312         future changes
313         are forward migrations in :mod:`muneem.migrations`, recorded in
314         ``schema_migrations``.
315         """
316         from muneem import migrations
317         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
318         baseline_version=SCHEMA_VERSION)
319
320     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
321         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
322         (sha256,)).fetchone()
323
324     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
325         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
326         txns."""
327         cols = ", ".join(fields)
328         placeholders = ", ".join("?" for _ in fields)
329         cur = conn.execute(
330             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
331             tuple(fields.values()),
332         )
333         rowid = cur.lastrowid # always set after a successful INSERT; guard for the type-
334         checker
335         if rowid is None: # pragma: no cover - defensive; INSERT always sets lastrowid
336             raise RuntimeError(f"INSERT into {table} did not return a row id")
337         return int(rowid)
338
339     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
340         """Insert into documents table. Delegates to the shared helper for DRY."""
341         return _insert_row(conn, "documents", dict(fields))
342
343     def record_provenance(
344         conn: sqlite3.Connection,
345         *,
346         document_id: int,
347         source_type: str,
348         parser: str | None = None,
349         parser_version: str | None = None,
350         mapping_layer: str | None = None,
351         confidence: str | None = None,
352     ) -> None:
353         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)`, so
355         a single
356         provenance record per document is reachable by join. Idempotent on ``document_id``
357         (a re-parse
358         overwrites rather than duplicating).
359         """
360         conn.execute(
361             "INSERT INTO provenance "

```

```

360         "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
361         "VALUES (?, ?, ?, ?, ?, ?) "
362         "ON CONFLICT(document_id) DO UPDATE SET "
363         "source_type=excluded.source_type, parser=excluded.parser, "
364         "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365         "confidence=excluded.confidence",
366         (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367     )
368
369
370     def record_parse(
371         conn: sqlite3.Connection,
372         document_id: int,
373         *,
374         confidence: str,
375         parser: str,
376         source_type: str = "pdf_statement",
377         parser_version: str | None = None,
378         mapping_layer: str = "deterministic",
379     ) -> None:
380         """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
381         parse)."""
382         conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
383         document_id))
384         record_provenance(
385             conn,
386             document_id=document_id,
387             source_type=source_type,
388             parser=parser,
389             parser_version=parser_version,
390             mapping_layer=mapping_layer,
391             confidence=confidence,
392         )
393
394     def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
395         return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
396         (document_id,)).fetchone()
397
398
399     def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
400         return _insert_row(conn, "offers", fields)
401
402     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
403         return conn.execute(
404             """
405             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
406                    d.external_id, d.path
407             FROM offers o
408             JOIN documents d ON d.id = o.document_id
409             ORDER BY o.id
410             """
411         ).fetchall()
412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415
416     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
417         return _insert_row(conn, "axis_cc_transactions", fields)
418
419
420     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->

```

```

list[sqlite3.Row]:
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

5. user (2026-06-10T16:22:53.577Z)

```

src\muneem\__init__.py
src\muneem\config.py
src\muneem\offers.py
src\muneem\ocr.py
src\muneem\extract.py
src\muneem\log.py
src\muneem\redaction.py
src\muneem\password_rules.py
src\muneem\secrets.py
src\muneem\statement_unlock.py
src\muneem\unlock.py
src\muneem\db_key.py
src\muneem\cli.py
src\muneem\recovery.py
src\muneem\errors.py
src\muneem\db.py
src\muneem\migrations.py

```

6. user (2026-06-10T16:22:54.654Z)

```

Found 7 files
src\muneem\db.py
src\muneem\parsers\hdfc.py
src\muneem\recovery.py
src\muneem\cli.py
src\muneem\db_key.py
src\muneem\secrets.py
src\muneem\parsers\savings.py

```

7. user (2026-06-10T16:22:58.630Z)

```
1      """Disaster recovery for the encrypted ledger ? a passphrase-wrapped recovery file.
2
3      The DB key lives in the OS keychain (fast, full-entropy). If that keychain is ever lost
4      ? OS
5      reinstall, new machine, profile/Credential-Manager corruption ? the ledger **and every
6      same-key
7      backup** become permanently unrecoverable. This module lets the user wrap that key under
8      a
9      passphrase they choose, into a recovery file they back up **off-machine**; with the file
10     *and* the
11     passphrase
12     they can restore the key into a fresh keychain.
13
14     Crypto: **Argon2id** (memory-hard KDF) derives a 256-bit wrapping key from the
15     passphrase + a random
16     salt; **AES-256-GCM** (authenticated) wraps the DB key. The recovery file holds only
17     ``{version, kdf params, salt, nonce, ciphertext}`` ? never the key, never the
18     passphrase. A wrong
19     passphrase fails the GCM authentication tag ? a clean error, with no decryption oracle.
20     """
21
22     from __future__ import annotations
23
24     import base64
25     import json
26     import os
27     from pathlib import Path
28     from typing import Any
29
30     from cryptography.exceptions import InvalidTag
31     from cryptography.hazmat.primitives.ciphers.aead import AESGCM
32     from cryptography.hazmat.primitives.kdf.argon2 import Argon2id
33
34     from muneem.errors import MuneemError
35
36     RECOVERY_VERSION = 1
37     _AAD = b"muneem-recovery-v1" # binds the ciphertext to this format/version
38
39     # Argon2id parameters (memory in KiB). Conservative-but-real; the file stores them so
40     they can be
41     # raised over time without breaking older recovery files.
42     _TIME_COST = 3
43     _MEMORY_KIB = 64 * 1024 # 64 MiB
44     _LANES = 4
45     _SALT_BYTES = 16
46     _NONCE_BYTES = 12
47     _KEY_BYTES = 32
48     _MIN_PASSPHRASE = 8
49
50     class RecoveryError(MuneemError):
51         """A recovery file could not be created, or could not be opened (wrong passphrase /
52         corrupt)."""
53
54     def _derive(passphrase: str, salt: bytes, *, time_cost: int, memory_kib: int, lanes:
55     int) -> bytes:
56         kdf = Argon2id(
57             salt=salt,
58             length=_KEY_BYTES,
59             iterations=time_cost,
60             lanes=lanes,
61             memory_cost=memory_kib,
```

```

55         )
56         return kdf.derive(passphrase.encode("utf-8"))
57
58
59     def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
60         """Return a JSON-serialisable recovery payload wrapping ``db_key`` under
61         ``passphrase``."""
62         if not passphrase or len(passphrase) < _MIN_PASSPHRASE:
63             raise RecoveryError(f"recovery passphrase must be at least {_MIN_PASSPHRASE}
64 characters")
65         salt = os.urandom(_SALT_BYTES)
66         nonce = os.urandom(_NONCE_BYTES)
67         key = _derive(passphrase, salt, time_cost=_TIME_COST, memory_kib=_MEMORY_KIB,
68 lanes=_LANES)
69         ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"), _AAD)
70         return {
71             "version": RECOVERY_VERSION,
72             "kdf": {
73                 "algo": "argon2id",
74                 "time_cost": _TIME_COST,
75                 "memory_kib": _MEMORY_KIB,
76                 "lanes": _LANES,
77             },
78             "salt": base64.b64encode(salt).decode(),
79             "nonce": base64.b64encode(nonce).decode(),
80             "ciphertext": base64.b64encode(ciphertext).decode(),
81         }
82
83     def unwrap_key(payload: dict[str, Any], passphrase: str) -> str:
84         """Recover the DB key from a recovery ``payload`` + ``passphrase``. Raises
85 RecoveryError."""
86         try:
87             salt = base64.b64decode(payload["salt"])
88             nonce = base64.b64decode(payload["nonce"])
89             ciphertext = base64.b64decode(payload["ciphertext"])
90             kdf = payload.get("kdf", {})
91         except (KeyError, ValueError, TypeError) as exc:
92             raise RecoveryError("recovery file is malformed") from exc
93         key = _derive(
94             passphrase,
95             salt,
96             time_cost=int(kdf.get("time_cost", _TIME_COST)),
97             memory_kib=int(kdf.get("memory_kib", _MEMORY_KIB)),
98             lanes=int(kdf.get("lanes", _LANES)),
99         )
100         try:
101             plain = AESGCM(key).decrypt(nonce, ciphertext, _AAD)
102         except InvalidTag as exc:
103             raise RecoveryError("wrong passphrase or corrupt recovery file") from exc
104         return plain.decode("utf-8")
105
106     def write_recovery_file(path: Path | str, db_key: str, passphrase: str) -> Path:
107         """Wrap ``db_key`` under ``passphrase`` and write the recovery file (owner-only
108 perms)."""
109         payload = wrap_key(db_key, passphrase)
110         out = Path(path)
111         out.parent.mkdir(parents=True, exist_ok=True)
112         out.write_text(json.dumps(payload, indent=2), encoding="utf-8")
113         try:
114             os.chmod(out, 0o600)
115         except OSError: # pragma: no cover - best-effort on platforms without POSIX perms
116             pass
117         return out

```

```

115
116
117     def read_recovery_file(path: Path | str, passphrase: str) -> str:
118         """Read a recovery file and return the unwrapped DB key. Raises RecoveryError on any
failure."""
119         p = Path(path)
120         try:
121             payload = json.loads(p.read_text(encoding="utf-8"))
122         except (OSError, ValueError) as exc:
123             raise RecoveryError(f"could not read recovery file {p.name}") from exc
124         return unwrap_key(payload, passphrase)
125

```

8. user (2026-06-10T16:22:59.932Z)

```

44:         with dbmod.connect_encrypted(db_path) as conn:
393:     from muneem.db_key import get_or_create_key
406:     n = dbmod.encrypt_database(db_path, get_or_create_key())
468:@db_app.command("setup-recovery")
469:def db_setup_recovery(
471:     None, help="Recovery file path (default: <data dir>/muneem-recovery.json).\"
474:     \"\"\"Create a passphrase-wrapped recovery file for the ledger's encryption key.
481:     from muneem import recovery
488:     out_path = out or (load_config().data_dir / \"muneem-recovery.json\")
489:     passphrase = getpass.getpass(\"New recovery passphrase: \")
494:     recovery.write_recovery_file(out_path, key, passphrase)
495:     except recovery.RecoveryError as exc:
499:         f\"wrote recovery file to {out_path}. Store it OFF this machine (password manager / \"
500:         \"separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a \"
501:         \"lost keychain means the ledger and its backups are unrecoverable.\"
505:@db_app.command(\"recover\")
506:def db_recover(
510:     \"\"\"Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain).\"\"\"
514:     from muneem import db_key, recovery
518:     key = recovery.read_recovery_file(file, passphrase)
519:     except recovery.RecoveryError as exc:
523:     # If the ledger is present, verify the recovered key actually opens it before committing
it.
529:         typer.echo(\"recovered a key, but it does not open the current ledger.\",
err=True)
532:     typer.echo(\"recovered the encryption key into the keychain.\")

```

9. user (2026-06-10T16:23:07.180Z)

```
.venv\Lib\site-packages\keyring\backends\windows.py
```

10. user (2026-06-10T16:23:10.802Z)

```

Exit code 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import keyring; print(keyring.__version__); print(keyring.get_keyring())
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: module 'keyring' has no attribute '__version__'
Name
----
.venv

```

11. user (2026-06-10T16:23:16.014Z)

```

1     from __future__ import annotations
2

```

```

3     import logging
4
5     from jaraco.context import ExceptionTrap
6
7     from ..backend import KeyringBackend
8     from ..compat import properties
9     from ..credentials import SimpleCredential
10    from ..errors import PasswordDeleteError
11
12    with ExceptionTrap() as missing_deps:
13        try:
14            # prefer pywin32-ctypes
15            from win32ctypes.pywin32 import pywintypes, win32cred
16
17            # force demand import to raise ImportError
18            win32cred.__name__ # noqa: B018
19        except ImportError:
20            # fallback to pywin32
21            import pywintypes
22            import win32cred
23
24            # force demand import to raise ImportError
25            win32cred.__name__ # noqa: B018
26
27    log = logging.getLogger(__name__)
28
29
30    class Persistence:
31        def __get__(self, keyring, type=None):
32            return getattr(keyring, '_persist', win32cred.CRED_PERSIST_ENTERPRISE)
33
34        def __set__(self, keyring, value):
35            """
36            Set the persistence value on the Keyring. Value may be
37            one of the win32cred.CRED_PERSIST_* constants or a
38            string representing one of those constants. For example,
39            'local machine' or 'session'.
40            """
41            if isinstance(value, str):
42                attr = 'CRED_PERSIST_' + value.replace(' ', '_').upper()
43                value = getattr(win32cred, attr)
44            keyring._persist = value
45
46
47    class DecodingCredential(dict):
48        @property
49        def value(self):
50            """
51            Attempt to decode the credential blob as UTF-16 then UTF-8.
52            """
53            cred = self['CredentialBlob']
54            try:
55                return cred.decode('utf-16')
56            except UnicodeDecodeError:
57                decoded_cred_utf8 = cred.decode('utf-8')
58                log.warning(
59                    "Retrieved a UTF-8 encoded credential. Please be aware that "
60                    "this library only writes credentials in UTF-16."
61                )
62                return decoded_cred_utf8
63
64
65    class WinVaultKeyring(KeyringBackend):
66        """
67        WinVaultKeyring stores encrypted passwords using the Windows Credential

```

```

68     Manager.
69
70     Requires pywin32
71
72     This backend does some gymnastics to simulate multi-user support,
73     which WinVault doesn't support natively. See
74     https://github.com/jaraco/keyring/issues/47#issuecomment-75763152
75     for details on the implementation, but here's the gist:
76
77     Passwords are stored under the service name unless there is a collision
78     (another password with the same service name but different user name),
79     in which case the previous password is moved into a compound name:
80     {username}@{service}
81     """
82
83     persist = Persistence()
84
85     @property
86     def priority(cls) -> float:
87         """
88         If available, the preferred backend on Windows.
89         """
90         if missing_deps:
91             raise RuntimeError("Requires Windows and pywin32")
92         return 5
93
94     @staticmethod
95     def _compound_name(username, service):
96         return f'{username}@{service}'
97
98     def get_password(self, service, username):
99         res = self._resolve_credential(service, username)
100         return res and res.value
101
102     def _resolve_credential(
103         self, service: str, username: str | None
104     ) -> DecodingCredential | None:
105         # first attempt to get the password under the service name
106         res = self._read_credential(service)
107         if not res or username and res['UserName'] != username:
108             # It wasn't found so attempt to get it with the compound name
109             res = self._read_credential(self._compound_name(username, service))
110         return res
111
112     def _read_credential(self, target):
113         try:
114             res = win32cred.CredRead(
115                 Type=win32cred.CRED_TYPE_GENERIC, TargetName=target
116             )
117         except pywintypes.error as e:
118             if e.winerror == 1168 and e.funcname == 'CredRead': # not found
119                 return None
120             raise
121         return DecodingCredential(res)
122
123     def set_password(self, service, username, password):
124         existing_pw = self._read_credential(service)
125         if existing_pw:
126             # resave the existing password using a compound target
127             existing_username = existing_pw['UserName']
128             target = self._compound_name(existing_username, service)
129             self._set_password(
130                 target,
131                 existing_username,
132                 existing_pw.value,

```



```

133         )
134         self._set_password(service, username, str(password))
135
136     def _set_password(self, target, username, password):
137         credential = dict(
138             Type=win32cred.CRED_TYPE_GENERIC,
139             TargetName=target,
140             UserName=username,
141             CredentialBlob=password,
142             Comment="Stored using python-keyring",
143             Persist=self.persist,
144         )
145         win32cred.CredWrite(credential, 0)
146
147     def delete_password(self, service, username):
148         compound = self._compound_name(username, service)
149         deleted = False
150         for target in service:
151             existing_pw = self._read_credential(target)
152             if existing_pw and existing_pw['UserName'] == username:
153                 deleted = True
154                 self._delete_password(target)
155         if not deleted:
156             raise PasswordDeleteError(service)
157
158     def _delete_password(self, target):
159         try:
160             win32cred.CredDelete(Type=win32cred.CRED_TYPE_GENERIC, TargetName=target)
161         except pywintypes.error as e:
162             if e.winerror == 1168 and e.funcname == 'CredDelete': # not found
163                 return
164             raise
165
166     def get_credential(self, service, username):
167         res = self._resolve_credential(service, username)
168         return res and SimpleCredential(res['UserName'], res.value)
169

```

12. user (2026-06-10T16:23:18.767Z)

25.7.0
keyring.backends.Windows.WinVaultKeyring (priority: 5)

13. user (2026-06-10T16:23:57.305Z)

```

27-     if not isinstance(keyring, backend.KeyringBackend):
28-         raise TypeError("The keyring must be an instance of KeyringBackend")
29-     _keyring_backend = keyring
30-
31-     --
52-         os.makedirs(root)
53-     except OSError:
54-         pass
55-     filename = os.path.join(root, 'keyringrc.cfg')
56-     if os.path.exists(filename):
57-         msg = f"Refusing to overwrite {filename}"
58-         raise RuntimeError(msg)
59-     with open(filename, 'w', encoding='utf-8') as file:
60-         file.write('[backend]\ndefault-keyring=keyring.backends.null.Keyring')
61-
62-
63: def get_password(service_name: str, username: str) -> str | None:
64-     """Get password from the specified service."""
65-     return get_keyring().get_password(service_name, username)

```

```

66-
67-
68: def set_password(service_name: str, username: str, password: str) -> None:
69-     """Set password for the user in the specified service."""
70-     get_keyring().set_password(service_name, username, password)
71-
72-
73-
144-     class_ = _load_keyring_class(keyring_name)
145-     # invoke the priority to ensure it is viable, or raise a RuntimeError
146-     class_.priority # noqa: B018
147-     return class_()
148-
149-
150-
153-         return load_keyring(os.environ['PYTHON_KEYRING_BACKEND'])
154-     except KeyError:
155-         return None
156-
157-
158-
163-     if not path.exists():
164-         raise FileNotFoundError(path)
165-     return path
166-
167-
168-
173-         config.read(_ensure_path(_config_path()), encoding='utf-8')
174-     except FileNotFoundError:
175-         return None
176-     _load_keyring_path(config)
177-
178-
179-
184-
185-     except (configparser.NoOptionError, ImportError):
186-         logger = logging.getLogger('keyring')
187-         logger.warning(
188-             "Keyring config file contains incorrect values.\n"
189-
190-
191-
200-         sys.path.insert(0, os.path.expanduser(path))
201-     except (configparser.NoOptionError, configparser.NoSectionError):
202-         pass

```

14. user (2026-06-10T16:23:58.303Z)

No matches found

15. user (2026-06-10T16:24:04.836Z)

```

15: class error(Exception):
16-     def __init__(self, *args, **kw):
17-         nargs = len(args)
18-         if nargs > 0:
19-             self.winerror = args[0]
20-         else:
21-             self.winerror = None
22-         if nargs > 1:
23-             self.funcname = args[1]
24-         else:
25-             self.funcname = None
26-         if nargs > 2:
27-             self.strerror = args[2]
28-         else:
29-             self.strerror = None
30-         Exception.__init__(self, *args, **kw)
31-
32-

```

```

33:@contextlib.contextmanager
34:def pywin32error():
35-    try:
36-        yield
37-    except WindowsError as exception:
38-        if not hasattr(exception, 'function'):
39-            exception.function = 'unknown'
40-        raise error(exception.winerror, exception.function, exception.strerror)
41-
42-
43-class datetime(_datetime):
44-
45-    def Format(self, fmt='%c'):
46-        return self.strftime(fmt)
47-
48-    --
49-
63:    except OSError as error:
64:        if error.errno == 22:
65:            raise ValueError(error.strerror)
66:        else:
67:            raise

```

16. user (2026-06-10T16:25:04.586Z)

```

1    import logging
2    from contextlib import closing
3
4    from jaraco.context import ExceptionTrap
5
6    from .. import backend
7    from ..backend import KeyringBackend
8    from ..compat import properties
9    from ..credentials import SimpleCredential
10   from ..errors import (
11       InitError,
12       KeyringLocked,
13       PasswordDeleteError,
14   )
15
16   try:
17       import secretstorage
18       import secretstorage.exceptions as exceptions
19   except ImportError:
20       pass
21   except AttributeError:
22       # See https://github.com/jaraco/keyring/issues/296
23       pass
24
25   log = logging.getLogger(__name__)
26
27
28   class Keyring(backend.SchemeSelectable, KeyringBackend):
29       """Secret Service Keyring"""
30
31       appid = 'Python keyring library'
32
33       @properties.classproperty
34       def priority(cls) -> float:
35           with ExceptionTrap() as exc:
36               secretstorage.__name__ # noqa: B018
37               if exc:
38                   raise RuntimeError("SecretStorage required")
39               if secretstorage.__version_tuple__ < (3, 2):
40                   raise RuntimeError("SecretStorage 3.2 or newer required")
41               try:
42                   with closing(secretstorage.dbus_init()) as connection:

```

```

43         if not secretstorage.check_service_availability(connection):
44             raise RuntimeError(
45                 "The Secret Service daemon is neither running nor "
46                 "activatable through D-Bus"
47             )
48     except exceptions.SecretStorageException as e:
49         raise RuntimeError(f"Unable to initialize SecretService: {e}") from e
50     return 5
51
52     def get_preferred_collection(self):
53         """If self.preferred_collection contains a D-Bus path,
54         the collection at that address is returned. Otherwise,
55         the default collection is returned.
56         """
57         bus = secretstorage.dbus_init()
58         try:
59             if hasattr(self, 'preferred_collection'):
60                 collection = secretstorage.Collection(bus, self.preferred_collection)
61             else:
62                 collection = secretstorage.get_default_collection(bus)
63         except exceptions.SecretStorageException as e:
64             raise InitError(f"Failed to create the collection: {e}.") from e
65         if collection.is_locked():
66             collection.unlock()
67             if collection.is_locked(): # User dismissed the prompt
68                 raise KeyringLocked("Failed to unlock the collection!")
69         return collection
70
71     def unlock(self, item):
72         if hasattr(item, 'unlock'):
73             item.unlock()
74         if item.is_locked(): # User dismissed the prompt
75             raise KeyringLocked('Failed to unlock the item!')
76
77     def get_password(self, service, username):
78         """Get password of the username for the service"""
79         collection = self.get_preferred_collection()
80         with closing(collection.connection):
81             items = collection.search_items(self._query(service, username))
82             for item in items:
83                 self.unlock(item)
84             return item.get_secret().decode('utf-8')
85
86     def set_password(self, service, username, password):
87         """Set password for the username of the service"""
88         collection = self.get_preferred_collection()
89         attributes = self._query(service, username, application=self.appid)
90         label = f"Password for '{username}' on '{service}'"
91         with closing(collection.connection):
92             collection.create_item(label, attributes, password, replace=True)
93
94     def delete_password(self, service, username):
95         """Delete the stored password (only the first one)"""
96         collection = self.get_preferred_collection()
97         with closing(collection.connection):
98             items = collection.search_items(self._query(service, username))
99             for item in items:
100                 return item.delete()
101         raise PasswordDeleteError("No such password!")
102
103     def get_credential(self, service, username):
104         """Gets the first username and password for a service.
105         Returns a Credential instance
106
107         The username can be omitted, but if there is one, it will use get_password

```

```
108         and return a SimpleCredential containing the username and password
109         Otherwise, it will return the first username and password combo that it finds.
110         """
111         scheme = self.schemes[self.scheme]
112         query = self._query(service, username)
113         collection = self.get_preferred_collection()
114
115         with closing(collection.connection):
116             items = collection.search_items(query)
117             for item in items:
118                 self.unlock(item)
119                 username = item.get_attributes().get(scheme['username'])
120                 return SimpleCredential(username, item.get_secret().decode('utf-8'))
```

17. user (2026-06-10T16:25:06.638Z)

```
5:class KeyringError(Exception):
6-    """Base class for exceptions in keyring"""
7-
8-    --
17:class InitError(KeyringError):
18-    """Raised when the keyring could not be initialised"""
19-
20-    --
21:class KeyringLocked(KeyringError):
22-    """Raised when the keyring failed unlocking"""
23-
24-    --
25:class NoKeyringError(KeyringError, RuntimeError):
26-    """Raised when there is no keyring backend"""
27-
```

18. user (2026-06-10T16:25:46.572Z)

Structured output provided successfully

19. assistant (2026-06-10T16:25:50.248Z)

Verification complete.